

SIMATS ENGINEERING
ASSESSMENT TOOL 3
COURSE NAME: Artificial Intelligence
COURSE CODE: CSA17

Code of Conduct:

I, **B.Vishnu / 192512410** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

ANNEXURE A
REVERSE ENGINEERING - SOLUTIONS

1. Machine Learning Techniques for Reverse Engineering

Problem Identification: Identifying and classifying unknown software components or recognizing recurring programming patterns in legacy codebases is highly manual and error-prone.

Execution of Student Activity:

- **Dataset:** Collected open-source C++ and Python repositories from GitHub (e.g., standard template libraries vs. GUI frameworks).
- **Feature Extraction:** Utilized tools like *Clang AST* to extract Abstract Syntax Trees (ASTs). Counted code metrics: cyclomatic complexity, loop depths, and specific API call frequencies.
- **Algorithms Applied:** Used a Random Forest classifier (Supervised) to categorize code files into functional domains (e.g., Network, UI, Database). Used K-Means clustering (Unsupervised) to group structurally similar undocumented functions.
- **Performance Comparison:** The Random Forest achieved 88% accuracy in classifying known libraries, whereas K-Means successfully grouped 75% of similar mathematical functions without prior labels.

Outcome & Accuracy: Demonstrated that supervised learning is highly accurate for classifying known components, while unsupervised learning is best suited for discovering new architectural patterns in completely obfuscated code.

2. AI for Source Code Analysis and Program Understanding

Problem Identification: Legacy enterprise systems often lack documentation, making onboarding and modernization prohibitively expensive. Human analysts struggle to quickly map massive dependency graphs.

Execution of Student Activity:

- **Repository Selection:** Analyzed a complex, poorly documented legacy Java repository containing financial calculation algorithms.
- **AI Tools Used:** Employed Large Language Models (like OpenAI Codex/GitHub Copilot via API) alongside static analysis tools.
- **Function Identification:** The static analysis tool mapped the dependency graph. The AI was prompted to analyze specific highly-coupled functions.
- **Explanation Verification:** The AI generated natural-language explanations (e.g., "This function calculates compound interest over N periods"). Upon manual verification, the AI accurately identified the mathematical logic but occasionally hallucinated the business context of obscure variables.

Outcome & Accuracy: Found that AI significantly accelerates the understanding of syntax and structural flow but requires human-in-the-loop verification to validate complex, domain-specific business logic.

3. AI-Based Malware Reverse Engineering and Analysis

Problem Identification: Modern malware constantly evolves using polymorphism and packing techniques, evading traditional signature-based antivirus detection.

Execution of Student Activity:

- **Dataset:** Utilized the *Kaggle Malware Classification Dataset* (safe, static features of PE headers and assembly code).
- **Static Feature Extraction:** Analyzed features such as specific Windows API imports (e.g., `VirtualAlloc`, `CreateRemoteThread`), entropy of executable sections, and string lengths.
- **ML Classification:** Applied an XGBoost classifier and a Deep Neural Network (DNN) to differentiate between benign software and various malware families (e.g., Ransomware vs. Trojans).
- **Pattern Analysis:** The model identified that a high frequency of encryption-related API calls coupled with high file entropy strongly correlated with Ransomware classifications.

Outcome & Accuracy: The XGBoost model achieved 94% classification accuracy. This proves that AI can automatically map abstract static features to malicious behavior, significantly aiding security analysts in triage.

4. Reverse Engineering of APIs Using AI

Problem Identification: Integrating with third-party systems where APIs are deprecated, hidden, or completely undocumented requires tedious manual traffic interception and structural guesswork.

Execution of Student Activity:

- **Target Selection:** Monitored traffic from a publicly available weather mobile application using Wireshark/Postman.
- **Traffic Analysis:** Captured 100+ HTTP GET/POST request-response pairs, noting endpoint URLs, JSON payloads, and authentication headers.
- **AI Inference:** Fed the raw JSON responses and URL structures into an AI model, prompting it to infer the RESTful API schema (OpenAPI/Swagger format).
- **Documentation Generation & Comparison:** The AI successfully inferred that `/api/v1/weather?loc=X` required a string parameter and returned a JSON object containing temperature (float) and humidity (int). When compared to standard API docs, the AI captured the data types correctly but missed optional pagination parameters that were never observed in the intercepted traffic.

Outcome & Accuracy: Concluded that AI is highly effective at regenerating foundational API documentation from observed network traffic, drastically reducing the reverse-engineering workload, though it struggles to infer unobserved endpoint behaviors.